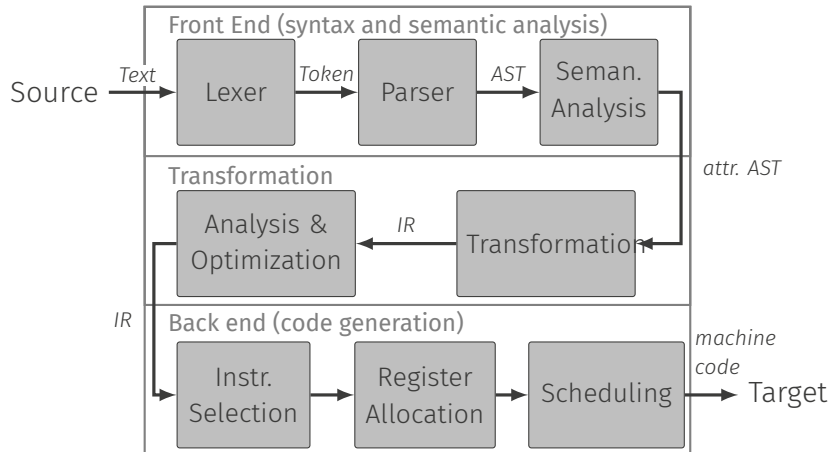


Control Flow Graphs

Roland Leißa (based on slides by S. Hack)

University of Göttingen

Overview



```
x = y + z * c + 5;
```

⇓

```
t1 = z * c;  
t2 = y + t1;  
x  = t2 + 5;
```

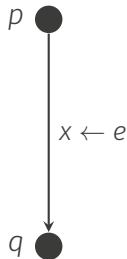
Intermediate Representations

- Each instruction in a function has a set of possible successor instructions in that function
- **Control flow graph (CFG)** is a directed graph (V, E, r) , $E \subseteq V^2, r \in V$:
 - V : instructions of the function
 - $(v, w) \in E$ iff w is a possible successor instruction of v
 - r : A root node which has no predecessors
- Need not be connected. **Unreachable code**: nodes not reachable from r
- Typically IRs are **intra-procedural**:
 - Each function/procedure is given by a **control flow graph**
 - Is a design choice
- Control flow between functions (call, return, exceptions, etc.) is not modelled explicitly in the CFG
 - In general undecidable what the contents of a function pointer are
 - Call/Return behavior follows stack principle (cannot express in CFG)

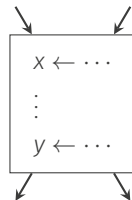
Basic Blocks

- It is common to summarize “straight-line” code segments of the CFG as **basic blocks**
- IRs typically use basic blocks as CFG nodes
- A basic block is a list of instructions with the following properties
 - the first instruction has multiple predecessors
 - the last instruction has multiple successors
 - every other instruction has one predecessor and one successor
- Reasons:
 - In many optimizations instructions with same control condition can be treated similarly (e.g. code motion, loop transformation, unreachable code elimination)
 - For some optimizations simpler special cases exist for straight-line code

Kinds of CFGs

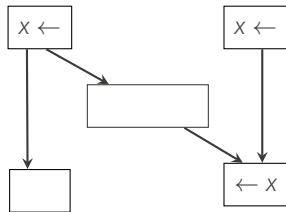
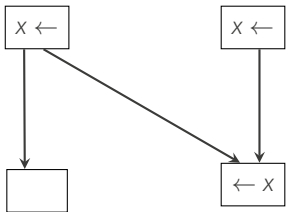


- Instructions on edges
- Nodes called program points
- Used in formal contexts (program analysis)



- Nodes are basic blocks
- Edges only denote control flow
- Close to actual implementation (BB = data structure, edge = pointer)

Critical Edges



- A CFG edge that goes from a block A with multiple successors to a block B with multiple predecessors is called **critical**
- Sometimes, the compiler needs to put code that is only executed if the program jumps from A to B
- To enable this, critical edges must be “broken”
- Loop back edges of **do** ... **while** loops and inverted loops are typically critical

Dominance

Let $G = (V, E, r)$ be a connected control flow graph.

Definition (Dominance)

A node v dominates a node w if all paths from r to w contain v . We write: $v \succeq w$.

Definition (Strict Dominance)

A node v **strictly** dominates a node w if $v \succeq w$ and $v \neq w$. We write: $v \succ w$.

There exist $O(|V|)$ algorithms for dominance. The most popular algorithm has $O(|V| \log^* |V|)$ and uses a union find data structure. A simple and practically efficient way is to use a data flow analysis approach.

Definition (Dominance Frontier)

The dominance frontier of a node v is the set of all nodes w_i such that v dominates an immediate predecessor of w_i , but $v \not\succeq w_i$.

It is the set of nodes where v 's dominance stops.

Theorem

Dominance is a partial order (i.e. reflexive, transitive, anti-symmetric).

Theorem

If $u \succeq w$ and $v \succeq w$ then either $u \succeq v$ or $v \succeq u$.

Corollary

All dominators of a node are totally ordered.

*Except for r every node v has a smallest **immediate** dominator $\text{idom}(v) \succ v$.*

$(V, \{(\text{idom}(v), v) \mid v \in V\})$ is a tree.

Dually, there is the notion of post-dominance. Consider a connected CFG with a unique exit node e such that e is the only node that has no successors.

Definition (Post Dominance)

A node w post-dominates a node v if w lies on all paths from v to e .

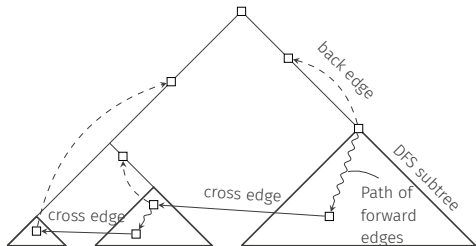
- Post-dominance equals dominance on $G^{\text{rev}} = (V, E^{\text{rev}})$ with $E^{\text{rev}} \triangleq \{(w, v) \mid (v, w) \in E\}$.
- A set of nodes dominated by some v and post-dominated by some w is called a **single entry single exit (SESE)** region of v and w
- The post-dominance frontier is called **control dependence**

Breadth-First Search (BFS)

```
std::queue<Node*> q;  
std::unordered_set<Node*> done;  
  
auto enqueue = [&](Node* n) {  
    if (done.emplace(n).second)  
        q.push(n);  
};  
  
enqueue(root);  
  
while (!q.empty()) {  
    n = q.top();  
    q.pop();  
    visit(n);  
  
    for (auto succ : n->succs())  
        enqueue(succ);  
}
```

Depth-First Search

```
public class DFS<V, G extends RootedGraph<V>> {  
    private final RootedGraph<V> graph;  
    private final Map<V, Integer> pre, post;  
    private int time = 0;  
  
    private void dfs(V node) {  
        pre.set(node, time++);  
        for (V w : graph.succs(node)) {  
            if (! pre.containsKey(w))  
                dfs(w);  
        }  
        post.set(node, time++);  
    }  
}
```



- Induces a tree T rooted at r
- Classifies each edge $(u, v) \in E$:
 - forward edge:** u is ancestor of v
 - back edge:** v is ancestor of u
 - cross edge:** otherwise

Classification depends on a concrete DFS (choice in order of visit of successors)

- The **DAG** of a CFG with respect to a **DFS** is obtained by removing all back edges
- **RPO (reverse post order):** Ordering nodes **reversely** by their post-visit time

Reducibility

A class of practically **important** and predominant CFGs is the class of reducible CFGs that are characterized below:

Definition

G is **reducible** if for every DFS tree, for each back edge (v, h) holds $h \succeq v$. We call a back edge (v, h) with $h \succeq v$ **back edge** and h a **header**.

Theorem

The following statements are equivalent:

1. G is reducible
2. G has a unique DAG
3. Every cycle in G has a node that dominates all other nodes in that cycle

Folk wisdom: In practice, almost every CFG is reducible

Loop Nesting

Definition

The (natural) loop of a back edge (v, h) consists of all nodes that reach v without going through h .

Theorem

For two loops L_1 and L_2 holds either:

1. $L_1 = L_2$
2. $L_1 \subseteq L_2$ or $L_2 \subseteq L_1$
3. $L_1 \cap L_2 = \emptyset$

Every cycle in a reducible CFG belongs to a natural loop.
Therefore, in reducible CFGs there is a clear loop nesting.

Summary: Classes of CFGs

- straight-line code
- **if** diamonds
- arbitrary forward edges
- reducible loops
- irreducible CFGs

In addition: critical edges?

- “A fast algorithm for finding dominators in a flowgraph”
<http://www.cs.princeton.edu/courses/archive/spr03/cs423/download/dominators.pdf>
- “A Simple, Fast Dominance Algorithm”
<https://www.cs.rice.edu/~keith/EMBED/dom.pdf>
- “On Loops, Dominators, and Dominance Frontiers”
<https://dl.acm.org/doi/abs/10.1145/358438.349330>
- “Controlled Node Splitting”
https://www.researchgate.net/publication/225518823_Controlled_Node_Splitting